Server-Factory-Core-Framework

Server Factory Core Framework

Tagline: The shared engine behind every Server Factory.

Summary: Core Framework is the Kotlin framework that underpins the Server Factory family of provisioning tools. It provides the common engine and abstractions that projects such as Mail Server Factory build upon, so each “factory” reuses one battle-tested foundation rather than re-implementing provisioning primitives.

Short description (~40 words): The shared Kotlin framework at the base of the Server Factory ecosystem. It supplies the common provisioning engine, connection abstractions, and installation-step machinery consumed by downstream factories (Mail Server Factory, Web Service Factory, SonarQube Factory, and others).

Long description (150–250 words): Core Framework is the foundational library of the Server-Factory organization: the reusable engine that the individual “factory” products (Mail Server Factory, Web Service Factory, SonarQube Factory, Caching Proxy Factory) all build on. The Server Factory approach is declarative — a user describes desired infrastructure as configuration, and a factory interprets it to install and initialize software on a target system — and Core Framework is where the common machinery for that pattern lives: the connection/transport abstractions, the installation-step model, and the shared plumbing that every factory needs. By centralizing this into one Kotlin framework, the family avoids duplicating provisioning logic across products and keeps behavior consistent: a connection type or installation primitive improved in Core Framework benefits every downstream factory. It is almost entirely Kotlin (roughly 990K bytes of Kotlin with a thin Shell layer), reflecting its role as a code library rather than a script collection. Downstream repos link back to it as their canonical dependency (Parallels-Utils, Qemu-Utils, Utils, and the Definitions packs all reference the Core

Framework repository as the hub of the ecosystem). Its README is intentionally minimal — it is infrastructure for other projects, versioned via `version.txt`/`version_code.txt` — and it predates the later AI work, making it part of the org's mature DevOps toolchain heritage.

Why we built it: Each provisioning tool needs the same core: ways to connect to targets and steps to install/configure software. Rebuilding that per product would fragment behavior and multiply bugs. Core Framework centralizes it so every factory shares one dependable engine.

Why it's a game-changer: It is the leverage point of the whole family — one framework improved once propagates correctness and features to every factory, embodying the "build once, reuse everywhere" philosophy in the infrastructure-automation domain.

What's innovative: - A single reusable provisioning framework abstracting connection + installation-step logic. - Clean separation between the engine (Core Framework) and product-specific factories. - Version-pinned distribution (`version.txt`/`version_code.txt`) for reproducible consumption.

Biggest technical challenges + how solved: - *Avoiding duplicated provisioning logic:* solved by extracting shared machinery into one framework consumed by all factories. - *Consistent behavior across products:* solved with common abstractions so connection types and steps behave identically everywhere. - *(UNVERIFIED)* Specific internal APIs are not documented in the public README; treat interface details as unverified beyond "shared framework consumed by the factories."

Tech stack (why + how): - **Kotlin** — the entire framework (~990K bytes); the language of the Server Factory family. - **Shell** — minimal supporting scripts. - **Gradle** — build toolchain (consistent with the family's `./gradlew` usage).

Public links: - GitHub: https://github.com/Server-Factory/Core-Framework (public; GitHub marks it a fork within the org). - Referenced as the hub by Mail-Server-Factory, Parallels-Utils, Qemu-Utils, Utils, and the Definitions repos.

Suggested diagrams/illustrations (OpenDesign): 1. Hub-and-spoke: Core Framework at center, factories (Mail/Web/SonarQube/Caching-Proxy) as spokes. 2. Layered stack: Core Framework → factory product → target system. 3. Shared-engine benefit: one fix in Core → propagates to all factories.

Site relevance: vasic.digital (infrastructure-automation heritage / reusable-framework story). Not AI-centric; present as the backbone of the provisioning toolchain.

Priority tier: serverfactory-tertiary

Source provenance: `gh repo view Server-Factory/Core-Framework` README (role as shared framework, version files); `gh api repos/Server-Factory/Core-Framework/languages` (Kotlin/Shell); cross-references in Parallels-Utils/Qemu-Utils/Utils/Definitions READMEs; `_analysis/github-helix-others.md` (org overview). Internal API specifics marked UNVERIFIED.